



人工智能算法与系统 课程实验报告

金融异常检测



学院 计算机科学与技术

专业 计算机技术

姓名 陈岳阳

学号 22521171

2025 年 11 月 19 日

目录

1 算法描述 3

 1.1 节点邻居采样 3

 1.2 邻居特征聚合 4

 1.3 节点表示更新与分类 4

2 算法性能分析 6

3 研究展望 8

图表

1 算法描述

在金融异常领域，图神经网络 (GNN) 因其能够充分利用节点之间的关系信息而受到广泛关注。本实验采用 GraphSAGE (Graph Sample and Aggregation)，通过在训练过程中对邻居节点进行采样并聚合特征，实现了对大规模图结构的高效建模。该算法最早由 Hamilton 等人提出，旨在克服传统图卷积网络 (GCN) 在处理大规模图时的计算瓶颈问题。GraphSAGE 的核心思想是对每个节点在每一层只采样固定数量的邻居节点，并通过聚合函数生成节点的表示，从而实现节点特征的逐层更新。

在本实验训练及测试使用 DGraph-Fin 数据集，一个针对金融交易异常分析构建的图数据集，包含大量节点及其复杂关系。模型输入为节点的原始特征向量，经过两层 GraphSAGE 网络进行特征聚合，使用 ReLU 激活函数增强表达能力。在每一层，节点特征首先与采样到的邻居节点特征进行拼接或加权聚合，然后通过线性映射更新节点表示。为了保持训练稳定性，实验中使用了梯度裁剪和 Dropout 技术，并使用 nll-Loss 优化分类函数。

训练过程中，GraphSAGE 通过 mini-batch 方式对节点进行采样，每个节点仅与其部分邻居交互，这种设计显著降低了内存消耗，同时保持了表示的有效性。验证与评估阶段，模型在验证集上计算 ROC-AUC 指标，以监控模型的泛化能力；在性能最优时保存模型权重，进一步用于对单个节点进行概率预测。整个算法流程不仅保证了大规模图的训练可行性，还能够针对节点级异常检测任务提供可靠的预测结果。

1.1 节点邻居采样

GraphSAGE 对每个训练节点采样固定的邻居，以控制计算复杂度。实验中采用固定邻居数的随机采样，同时在聚合中包含自身节点。

```
1  def _get_unique_neighs_list(self, nodes, num_sample=10):  
2      """  
3      从 SparseTensor 高效采样邻居，复杂度  $O(B \times d)$   
4      """  
5      adj_t = self.adj_lists  
6      nodes = nodes.tolist() if hasattr(nodes, "tolist") else nodes  
7  
8      rowptr, col, _ = adj_t.csr()  
9      samp_neighs = []  
10     for node in nodes:  
11         start, end = rowptr[node].item(), rowptr[node + 1].item()  
12         neigh = col[start:end].tolist()
```

```

13     s = set(random.sample(neigh, num_sample)) if len(neigh) >=
        num_sample else set(neigh)
14     s.add(node) # 包含自身
15     samp_neighs.append(s)
16
17     unique_nodes_list = list(set().union(*samp_neighs))
18     unique_nodes = {nid: idx for idx, nid in
        enumerate(unique_nodes_list)}
19     return samp_neighs, unique_nodes, unique_nodes_list

```

1.2 邻居特征聚合

对采样到的邻居节点特征进行加权求和或均值聚合，得到当前节点的嵌入表示。GraphSAGE 提供多种聚合函数。本实验中，仅实现并使用均值聚合函数。

```

1  def aggregate(self, nodes, pre_hidden_embs, pre_neighs,
    num_sample=10):
2      unique_nodes_list, samp_neighs, unique_nodes = pre_neighs
3      if not self.gcn:
4          samp_neighs = [(s - set([nodes[i]])) for i, s in
        enumerate(samp_neighs)]
5      embed_matrix = pre_hidden_embs[torch.LongTensor(unique_nodes_list)]
6      mask = torch.zeros(len(samp_neighs), len(unique_nodes))
7      column_indices = [unique_nodes[n] for samp_neigh in samp_neighs for n
        in samp_neigh]
8      row_indices = [i for i in range(len(samp_neighs)) for _ in
        range(len(samp_neighs[i]))]
9      mask[row_indices, column_indices] = 1
10     mask = mask.div(mask.sum(1, keepdim=True))
11     aggregate_feats = mask.mm(embed_matrix)
12     return aggregate_feats

```

1.3 节点表示更新与分类

将节点自身特征与聚合特征拼接或替换，通过线性变换与激活函数得到新的节点嵌入，并通过分类层输出节点类别概率。整个 GraphSAGE 的前向传播过程通过逐层聚合节点邻居特征实现。

```
1 class SageLayer(nn.Module):
2     def forward(self, self_feats, aggregate_feats, neighs=None):
3         combined = torch.cat([self_feats, aggregate_feats], dim=1) if not
4         self.gcn else aggregate_feats
5         combined = F.relu(self.weight.mm(combined.t())).t()
6         return combined
7
8 class Classification(nn.Module):
9     def forward(self, embeds):
10         logits = torch.log_softmax(self.layer(embeds), 1)
11         return logits
12
13 def forward(self, nodes_batch):
14     # 构建逐层采样邻居列表
15     ...
16     # 特征聚合
17     for index in range(1, self.num_layers + 1):
18         aggregate_feats = self.aggregate(nb, pre_hidden_embs, pre_neighs)
19         cur_hidden_embs = sage_layer(self_feats=pre_hidden_embs[nb],
20         aggregate_feats=aggregate_feats)
21         pre_hidden_embs = cur_hidden_embs
22
23     # 输出分类概率
24     return self.layers[-1](pre_hidden_embs)
```

2 算法性能分析

在本实验中，GraphSAGE 模型在 DGraph-Fin 数据集上进行了金融异常分析。模型采用两层结构，每层隐藏维度为 64，每个节点随机采样固定数量的邻居节点进行聚合，以捕捉节点间的局部图结构信息。训练优化器选择 Adam，学习率为 $5e-4$ ，同时进行梯度裁剪以防止梯度爆炸。模型在训练阶段的核心代码如下：

```
1  def train_one_epoch(data, graphsage, batch_size, device,
2      optimizer):
3      graphsage.train()
4      train_idx = data.train_mask
5      perm = torch.randperm(train_idx.size(0), device=device)
6      train_idx = train_idx[perm]
7      labels = data.y.to(device)
8
9      total_loss = 0.0
10     batches = math.ceil(len(train_idx) / batch_size)
11     for index in range(batches):
12         start = index * batch_size
13         end = (index + 1) * batch_size
14         nodes_batch = train_idx[start:end]
15         labels_batch = labels[nodes_batch]
16
17         logits = graphsage(nodes_batch)
18         loss = F.nll_loss(logits, labels_batch)
19         total_loss += loss.item()
20
21     optimizer.zero_grad()
22     loss.backward()
23     nn.utils.clip_grad_norm_(list(graphsage.parameters()), 5)
24     optimizer.step()
25     return total_loss / batches
```

在每轮实验完成后，模型会在验证集上计算 ROC-AUC 指标，并保存性能最优的模型权重。

```
1  @torch.no_grad()
2  def evaluate(data, model, batch_size, device, max_val_auc):
3      model.eval()
4      val_idx = data.valid_mask
5      val_probs = torch.zeros(len(val_idx), device=device)
```

```
6     for i in range(0, len(val_idx), batch_size):
7         batch = val_idx[i:i+batch_size]
8         logits = model(batch)
9         val_probs[i:i+batch_size] = logits.exp()[ :, 1]
10
11     val_labels = data.y[val_idx].to(device)
12     val_auc = roc_auc_score(val_labels.cpu(), val_probs.cpu())
13     if val_auc > max_val_auc:
14         torch.save(model.state_dict(), './results/model_best.pt')
15         max_val_auc = val_auc
16     return max_val_auc
```

实验结果显示，GGraphSAGE 在 DGraph-Fin 数据集上最佳验证 AUC 达到 0.77，明显由于未使用图结构信息的 MLP 模型。这种性能提升主要源于 GraphSAGE 的邻居采样与特征聚合机制，使每个节点在表示学习过程中不仅依赖自身特征，也能够综合邻居节点信息。

3 研究展望

尽管本实验中基于 GraphSAGE 的模型在金融异常检测任务中展现了较好的性能，但仍存在一定的优化空间与研究方向。首先，当前模型在邻居节点采样时采用固定数量的随机采样策略，这种方法在稀疏图或极度不均衡图中可能导致部分关键节点特征被忽略。未来研究可考虑引入自适应采样机制，根据节点的重要性或连接强度动态调整邻居采样数量，从而更精确地捕捉局部结构信息。

其次，GraphSAGE 在聚合邻居特征时主要使用均值或拼接操作，缺少对邻居特征的权重学习能力。引入注意力机制（Graph Attention）可以赋予模型对不同邻居节点的重要性进行动态加权，从而提高异常节点的识别能力。此外，可以考虑结合多层次图嵌入方法，对局部与全局图结构进行联合建模，增强模型对复杂金融关系的捕捉能力。

在模型训练与优化方面，目前的实现使用了单一的交叉熵损失函数和固定学习率优化策略。未来可尝试多任务学习框架，将异常检测与节点分类、社区检测等任务联合训练，利用辅助任务改善主任务的泛化能力。同时，自监督或对比学习方法也可以在缺乏标签的数据场景下增强图表示的质量，降低对人工标注的依赖。

最后，从应用角度来看，金融异常检测对实时性要求较高，而当前 GraphSAGE 模型在大规模图上仍存在计算瓶颈。未来研究可以探索图分块技术、分布式训练以及模型蒸馏策略，在保证准确性的同时显著提升推理速度，从而实现实时金融交易数据的在线监控与异常预警。

综上所述，GraphSAGE 在金融异常分析中的应用具有较强的潜力，但仍有多方面的改进空间，包括邻居采样策略、特征聚合机制、训练优化方法以及大规模图的高效推理。这些方向的探索不仅有助于提升模型性能，也将为图神经网络在金融风控领域的广泛应用提供理论与实践支持。